

Lesson 7 - Quick Reference Card

□ One to One

```
// User Model
public function profile()
{
    return $this->hasOne(Profile::class);
}

// Profile Model
public function user()
{
    return $this->belongsTo(User::class);
}

// Usage
$user = User::find(1);
$profile = $user->profile;

$profile = Profile::find(1);
$user = $profile->user;

// Create
$user->profile()->create(['bio' => 'Developer']);
```

□ One to Many

```
// User Model
public function posts()
{
    return $this->hasMany(Post::class);
}

// Post Model
public function user()
{
    return $this->belongsTo(User::class);
}
```

```
// Usage
$user = User::find(1);
$posts = $user->posts;
$posts = $user->posts()->where('published', true)->get();

$post = Post::find(1);
$user = $post->user;

// Create
$user->posts()->create(['title' => 'Title']);
$user->posts()->createMany([...]);
```

□ Many to Many

```
// Post Model
public function tags()
{
    return $this->belongsToMany(Tag::class);
}

// Tag Model
public function posts()
{
    return $this->belongsToMany(Post::class);
}

// Usage
$post = Post::find(1);
$tags = $post->tags;

// Attach
$post->tags()->attach(1);
$post->tags()->attach([1, 2, 3]);

// Detach
$post->tags()->detach(1);
$post->tags()->detach(); // All

// Sync
$post->tags()->sync([1, 2, 3]);
$post->tags()->syncWithoutDetaching([4, 5]);

// Toggle
```

```
$post->tags()->toggle([1, 2]);
```

▣ Pivot with Additional Data

```
// Post Model
public function tags()
{
    return $this->belongsToMany(Tag::class)
        ->withPivot('order', 'notes')
        ->withTimestamps();
}

// Attach with data
$post->tags()->attach(1, ['order' => 1]);

// Access data
foreach ($post->tags as $tag) {
    echo $tag->pivot->order;
    echo $tag->pivot->created_at;
}

// Sync with data
$post->tags()->sync([
    1 => ['order' => 1],
    2 => ['order' => 2],
]);
```

▣ Has Many Through

```
// Country Model
public function posts()
{
    return $this->hasManyThrough(
        Post::class,    // Final model
        User::class,    // Intermediate model
        'country_id',   // FK on users
        'user_id',      // FK on posts
        'id',           // Local key on countries
    );
}
```

```

        'id'          // Local key on users
    );
}

// Usage
$country = Country::find(1);
$posts = $country->posts;

```

❑ Polymorphic Relations

```

// Migration
$table->morphs('commentable'); // commentable_id + commentable_type

// Comment Model
public function commentable()
{
    return $this->morphTo();
}

// Post Model
public function comments()
{
    return $this->morphMany(Comment::class, 'commentable');
}

// Video Model
public function comments()
{
    return $this->morphMany(Comment::class, 'commentable');
}

// Usage
$post = Post::find(1);
$comments = $post->comments;

$comment = Comment::find(1);
$commentable = $comment->commentable; // Post or Video

// Create
$post->comments()->create(['content' => 'Great!']);

```

▣ Many to Many Polymorphic

```
// Tag Model
public function posts()
{
    return $this->morphedByMany(Post::class, 'taggable');
}

public function videos()
{
    return $this->morphedByMany(Video::class, 'taggable');
}

// Post Model
public function tags()
{
    return $this->morphToMany(Tag::class, 'taggable');
}

// Usage
$post->tags()->attach([1, 2, 3]);
$tag = Tag::find(1);
$posts = $tag->posts;
$videos = $tag->videos;
```

⚡ Eager Loading

```
// Load single relationship
$posts = Post::with('user')->get();

// Load multiple relationships
$posts = Post::with(['user', 'comments', 'tags'])->get();

// Load nested
$posts = Post::with('user.profile')->get();
$posts = Post::with('comments.user')->get();

// Load multiple nested
```

```

$post = Post::with([
    'user.profile',
    'comments.user',
    'tags'
])->get();

// Load with constraints
$post = Post::with(['comments' => function ($query) {
    $query->where('approved', true)
        ->latest()
        ->take(5);
}])->get();

// Lazy Eager Loading
$post = Post::all();
$post->load('user');
$post->loadMissing('user'); // If not loaded

// Count relationships
$post = Post::withCount('comments')->get();
echo $post[0]->comments_count;

// Count with constraints
$post = Post::withCount([
    'comments',
    'comments as approved_comments_count' => function ($query) {
        $query->where('approved', true);
    }
])->get();

```

□ Querying Relationships

```

// whereHas - Query with condition
$post = Post::whereHas('user', function ($query) {
    $query->where('country_id', 1);
})->get();

$post = Post::whereHas('comments', function ($query) {
    $query->where('approved', true);
})->get();

// doesntHave - Doesn't have
$post = Post::doesntHave('comments')->get();

```

```
// has - Has
$posts = Post::has('comments')->get();
$posts = Post::has('comments', '>=', 5)->get();

// orWhereHas
$posts = Post::whereHas('tags', function ($query) {
    $query->where('name', 'Laravel');
})->orWhereHas('tags', function ($query) {
    $query->where('name', 'PHP');
})->get();
```

📦 Quick Examples

```
// Post with all relationships
$post = Post::with([
    'user.profile',
    'comments.user',
    'tags'
])->find(1);

// Users with post count
$users = User::withCount('posts')
    ->orderBy('posts_count', 'desc')
    ->get();

// Posts without comments
$posts = Post::doesntHave('comments')->get();

// Posts from specific country
$posts = Post::whereHas('user', function ($query) {
    $query->where('country_id', 1);
})->get();

// Statistics
$user = User::withCount(['posts', 'comments'])->find(1);
echo "Posts: " . $user->posts_count;
echo "Comments: " . $user->comments_count;
```

❑ Migration Patterns

```
// One to Many
$table->foreignId('user_id')->constrained()->onDelete('cascade');

// Many to Many (Pivot Table)
Schema::create('post_tag', function (Blueprint $table) {
    $table->id();
    $table->foreignId('post_id')->constrained()->onDelete('cascade');
    $table->foreignId('tag_id')->constrained()->onDelete('cascade');
    $table->timestamps();
    $table->unique(['post_id', 'tag_id']);
});

// Polymorphic
$table->morphs('commentable'); // commentable_id + commentable_type

// Or manually
$table->unsignedBigInteger('commentable_id');
$table->string('commentable_type');
$table->index(['commentable_id', 'commentable_type']);
```

❑ Best Practices

❑ Always use Eager Loading ❑ Use withCount for statistics ❑ Specify required columns:
with('user:id,name') ❑ Use whereHas to query relationships

❑ Don't forget Eager Loading (N+1 problem) ❑ Don't use get() twice ❑ Don't forget to define belongsTo

❑ Quick Links

- [Main Lesson](#)
- [Previous Lesson](#)
- [Next Lesson](#)